

Gilifast

Database Reference

PostgreSQL via Prisma · 38 tables · 41 enums

Generated from `prisma/schema.prisma` on 15 August 2026 (WITA)

Commit `f4cf58c` · branch `claude/boat-ticketing-mvp-YAGl9`

This document is generated, not written

Re-run `scripts/schema-docs.py` after any schema change. Editing this PDF by hand guarantees it disagrees with the database on the next migration.

Reading this document

Every table below is generated directly from the Prisma schema. A `*` after a column name means it is required (Prisma `NOT NULL`); everything else is nullable. The `Notes` column carries the schema's own comments, which is where the reasoning behind a column usually lives.

Table names are quoted PascalCase in Postgres. Prisma does not remap them, so raw SQL needs double quotes: `SELECT * FROM "Booking"`, not `FROM booking`.

Conventions that apply across the schema

Identifiers. Every table uses a `cuid()` string primary key, not a sequence. IDs are therefore safe to generate client-side and carry no ordering information — never sort by `id` expecting chronology; use `createdAt`.

Money. All amounts are `Decimal`, never `float`, and all are IDR unless a sibling currency column says otherwise. The application carries `Prisma.Decimal` end to end to avoid rounding drift; see `lib/pricing.ts`.

Time. Timestamps are stored UTC. Every customer-facing time is rendered in WITA (Asia/Makassar) through `lib/datetime.ts`. A raw timestamp read out of the database is eight hours behind what the passenger was told.

Soft deletes. `Operator`, `Boat`, `Schedule`, `Port`, `OperatorStaff` and `TravelAgent` carry `deletedAt`. Every list query must filter `deletedAt: null` — a query that forgets it silently resurrects retired boats into search results.

Tenant scoping. Operator-facing queries are scoped by `operatorId`; logged-in customer queries by `customerId`. Admin queries are unscoped on purpose. There is no row-level security in the database — scoping is enforced entirely in application code, so an unscoped query is a cross-tenant data leak, not just a bug.

Foreign keys into the catalogue are `onDelete: Restrict`

Deleting a row means walking its subtree bottom-up by hand. This is deliberate: it makes it impossible to erase a boat and take a year of bookings with it. `lib/operator-purge.ts` is the only code that does this, and it refuses whenever the subtree contains a `Payment`, `Refund` or `Ticket` — no row in the financial chain is ever destroyed.

Three behaviours the column list does not show

These are the parts of the data model most likely to surprise someone editing the database directly.

1 • Schedule is a template; Leg is a frozen sale price

`Schedule.basePrice` is the pricing policy for a route. When a concrete departure is generated, `lib/legs.ts` copies that value into the new `Leg.basePrice`. From then on the two are independent numbers, and booking reads only the leg (`lib/booking-engine.ts`).

The freeze is deliberate: it keeps the price stable for departures already on sale, and it is what makes a historical booking auditable. The yield multiplier still reads live from `Schedule.pricingTiers`, because two departures of the same schedule can be at very different occupancy.

Changing a price means updating both tables

Updating `Schedule.basePrice` alone changes nothing customers can see — the nightly `topup-legs` cron only adds new dates, never rewrites existing legs, so the old price keeps selling for up to the full 60-day horizon. Updating `Leg` alone reprices what is on sale, but the next cron run generates tomorrow's departure at the old price again.

2 • Payment settling does not issue a ticket

Money arriving moves a booking to `AWAITING_CONFIRMATION` and mints no `Ticket` rows. Operators on this platform take bookings by phone and do not use the dashboard, so an admin confirms the boat is running before any boarding pass exists. Tickets are created on the far side of that call, by `issueTicketsForBooking` in `lib/ticket-issuer.ts`.

Seats stay held on the `Leg` for the whole waiting window, and expiry only sweeps `PENDING_PAYMENT`, so a paid booking cannot have its seats reclaimed underneath it.

3 • Two enums carry values the application no longer writes

`SalesChannel.GILIJET` is the pre-rename value for `GILIFAST`, and `PaymentProvider` retains `XENDIT`, `MAYAR` and `STRIPE`. They are kept because existing rows reference them, and dropping an in-use enum value makes `prisma db push` refuse the entire push. `lib/sales-channel.ts` folds the legacy channel onto the current one for every report and display; anything asking "was this sold on our own platform?" must match both.

No migration history. There is no `prisma/migrations/` directory. Schema reaches the database through `prisma db push` during the build, run by `scripts/db-push-if-configured.mjs`, never with `--accept-data-loss`. A destructive change fails the build rather than executing — including any table created by hand in the SQL editor that the schema does not know about.

Tables

38 tables, grouped as the schema groups them.

Users

Customer

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
email *	String	unique	
passwordHash	String?		
googleId	String?	unique	
fullName *	String		
phoneNumber	String?		
nationality	String?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
bookings	Booking	has many	inverse side	—
reviews	Review	has many	inverse side	—
loyaltyAccount	LoyaltyAccount	has one (optional)	inverse side	—

Indexes: @@index([email])

Operator

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
email *	String	unique	
passwordHash *	String		
companyName *	String		
contactPerson *	String		
phoneNumber *	String		
bankAccountInfo *	Json		{ bankName, accountNumber, accountHolder }
npwp	String?		Indonesian tax ID
status *	OperatorStatus	default PENDING	
documentsVerified *	Boolean	default false	
commissionRate *	Decimal	default 0.08, Decimal(5, 4)	
agentId	String?		
deletedAt	DateTime?		
createdAt *	DateTime	default now()	

Column	Type	Constraints	Notes
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
boats	Boat	has many	inverse side	—
legs	Leg	has many	inverse side	—
bookings	Booking	has many	inverse side	—
services	OperatorServicesOnOperator	has many	inverse side	—
staff	OperatorStaff	has many	inverse side	—
documents	OperatorDocument	has many	inverse side	—
travelAgents	TravelAgent	has many	inverse side	—
cashDrawerSessions	CashDrawerSession	has many	inverse side	—
notifications	OperatorNotification	has many	inverse side	—
agent	Agent	has one (optional)	agentId	—

Indexes: @@index([status])

Admin

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
email *	String	unique	
passwordHash *	String		
fullName *	String		
role *	AdminRole	default STAFF	
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Boats & Schedules

Boat

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
name *	String		
registrationNumber *	String	unique	
capacity *	Int		
photos *	String[]		URLs
description	String?		
status *	BoatStatus	default ACTIVE	

Column	Type	Constraints	Notes
deletedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	Restrict
schedules	Schedule	has many	inverse side	—
facilities	BoatFacilitiesOnBoat	has many	inverse side	—
positions	BoatPosition	has many	inverse side	—

Indexes: @@index([operatorId])

Schedule

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
boatId *	String		
originPort *	String		
destinationPort *	String		
originPortId	String?		
destinationPortId	String?		
departureTime *	String		"HH:MM" local time (Asia/Makassar by default)
durationMinutes *	Int		
basePrice *	Decimal	Decimal(12, 2)	IDR
daysOfWeek *	Int[]		1=Mon ... 7=Sun
status *	ScheduleStatus	default ACTIVE	
pricingTiers	Json?		
fareMatrix	Json?		Full operator price sheet: season x passenger category x one-way/return. basePrice holds the adult/low/one-way cell; this keeps the rest. See lib/fares.ts.
deletedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
boat	Boat	has one	boatId	Restrict
legs	Leg	has many	inverse side	—
reviews	Review	has many	inverse side	—
transitStops	TransitStop	has many	inverse side	—
originPortRel	Port	has one (optional)	originPortId	—

Relation	Target	Cardinality	Foreign key	On delete
destinationPortRel	Port	has one (optional)	destinationPortId	—

Indexes: @@index([boatId, status]) @@index([originPort, destinationPort])

Legs (Specific Departures)

Leg

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
scheduleId *	String		
operatorId *	String		
departureDate *	DateTime		UTC instant of this specific departure
totalCapacity *	Int		
availableSeats *	Int		
basePrice *	Decimal	Decimal(12, 2)	
status *	LegStatus	default OPEN	
cancellationReason	String?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
schedule	Schedule	has one	scheduleId	Restrict
operator	Operator	has one	operatorId	Restrict
bookings	Booking	has many	inverse side	—
reschedulesAsOriginal	RescheduleRequest	has many	inverse side	—
reschedulesAsRequested	RescheduleRequest	has many	inverse side	—
seatHolds	SeatHold	has many	inverse side	—
weatherSnapshots	WeatherSnapshot	has many	inverse side	—

Indexes: @@unique([scheduleId, departureDate]) @@index([departureDate, status]) @@index([operatorId, departureDate])

Bookings & Tickets

Booking

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
bookingReference *	String	unique	
legId *	String		

Column	Type	Constraints	Notes
customerId	String?		
operatorId *	String		
agentId	String?		
salesChannel *	SalesChannel	default GILIFAST	
salesStaffId	String?		
salesAgentId	String?		
agentCommissionAmount	Decimal?	Decimal(14, 2)	
agentCommissionPercent	Decimal?	Decimal(5, 4)	
pph23WithholdingAmount	Decimal?	Decimal(14, 2)	
cashCollected	Decimal?	Decimal(14, 2)	
externalAggregatorName	String?		
externalAggregatorRef	String?		
eFakturNumber	String?		
eFakturStatus *	EFakturStatus	default NOT_REQUIRED	
customerName *	String		
customerEmail *	String		
customerPhone *	String		
customerNationality	String?		
totalAmount *	Decimal	Decimal(12, 2)	
commissionAmount *	Decimal	Decimal(12, 2)	
operatorAmount *	Decimal	Decimal(12, 2)	
currency *	String	default "IDR"	
status *	BookingStatus	default PENDING_PAYMENT	
refundDeadline *	DateTime		
refundPolicySnapshot	Json?		
paymentGatewayRef	String?		
idempotencyKey	String?	unique	
notes	String?		
promotionId	String?		
discountAmount *	Decimal	default 0, Decimal(12, 2)	
reminderSentAt	DateTime?		
availabilityDecidedAt	DateTime?		Manual availability check: who rang the operator, when, and what they were told. availabilityDecidedAt is set on both approve and reject; the booking status says which way it went.
availabilityDecidedById	String?		
availabilityNote	String?		
createdAt *	DateTime	default now()	

Column	Type	Constraints	Notes	
updatedAt [*]	DateTime	auto-updated		
Relations				
Relation	Target	Cardinality	Foreign key	On delete
leg	Leg	has one	legId	Restrict
customer	Customer	has one (optional)	customerId	SetNull
operator	Operator	has one	operatorId	Restrict
agent	Agent	has one (optional)	agentId	—
salesStaff	OperatorStaff	has one (optional)	salesStaffId	—
salesAgent	TravelAgent	has one (optional)	salesAgentId	—
promotion	Promotion	has one (optional)	promotionId	Restrict
promotionRedemption	PromotionRedemption	has one (optional)	inverse side	—
tickets	Ticket	has many	inverse side	—
payment	Payment	has one (optional)	inverse side	—
refund	Refund	has one (optional)	inverse side	—
review	Review	has one (optional)	inverse side	—
rescheduleRequest	RescheduleRequest	has one (optional)	inverse side	—
addons	BookingAddon	has many	inverse side	—
seatHold	SeatHold	has one (optional)	inverse side	—
loyaltyTransactions	LoyaltyTransaction	has many	inverse side	—
Indexes: @@index([legId]) @@index([status]) @@index([customerEmail]) @@index([customerId]) @@index([promotionId]) @@index([operatorId, status]) @@index([operatorId, createdAt]) @@index([operatorId, salesChannel, createdAt]) @@index([salesAgentId])				

Ticket

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
ticketCode *	String	unique	TK-2026-05-000001-1
bookingId *	String		
passengerName *	String		
passengerIdNumber	String?		
qrHash *	String	unique	
status *	TicketStatus	default ISSUED	
checkedInAt	DateTime?		
checkedInById	String?		
checkedInBy	String?		
createdAt *	DateTime	default now()	
Relations			

Relation	Target	Cardinality	Foreign key	On delete
booking	Booking	has one	bookingId	Restrict
checkedInByStaff	OperatorStaff	has one (optional)	checkedInById	—

Indexes: @@index([bookingId]) @@index([qrHash]) @@index([checkedInById])

Payments & Refunds

Payment

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
bookingId *	String	unique	
amount *	Decimal	Decimal(12, 2)	
method *	PaymentMethod		
status *	PaymentStatus	default PENDING	
gatewayProvider *	PaymentProvider	default MIDTRANS	
gatewayReference	String?		
gatewayFee *	Decimal	default 0, Decimal(12, 2)	
paidAt	DateTime?		
failedReason	String?		
rawWebhookData	Json?		
instrumentData	Json?		Midtrans Snap: the snap token stored as gateway reference so the pay page can use it to re-open the payment popup.
expiresAt	DateTime?		
presentmentCurrency	String?	VarChar(3)	Cross-currency capture. `amount` above stays the authoritative IDR total; these record what the customer was actually charged abroad. PayPal cannot settle IDR, so a PayPal payment always charges a presentment currency and a refund must return the presentment amount captured — re-converting at a later rate would silently over- or under-refund.
presentmentAmount	Decimal?	Decimal(12, 2)	
fxRate	Decimal?	Decimal(18, 6)	IDR per 1 presentment unit
fxQuotedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
booking	Booking	has one	bookingId	Restrict

Indexes: @@index([status]) @@index([gatewayReference])

Refund

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
bookingId *	String	unique	
originalAmount *	Decimal	Decimal(12, 2)	
refundAmount *	Decimal	Decimal(12, 2)	
reason *	RefundReason		
status *	RefundStatus	default PENDING	
gatewayReference	String?		
processedAt	DateTime?		
approvedBy	String?		Admin.id who approved/rejected
approvedAt	DateTime?		
adminNote	String?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
booking	Booking	has one	bookingId	Restrict

Indexes: @@index([status])

Webhook Events (Retry Queue)

WebhookEvent

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
provider *	String		'xendit' 'midtrans'
eventType *	String		'invoice.paid' 'invoice.expired' etc.
externalRef *	String		bookingReference or refund gateway id
rawPayload *	Json		
status *	WebhookEvents tatus	default PENDING	
attempts *	Int	default 0	
lastAttemptAt	DateTime?		
processedAt	DateTime?		
errorMessage	String?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Indexes: @@index([status, createdAt]) @@index([provider, externalRef])

Promotions

Promotion

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
code *	String	unique	
discountType *	DiscountType		
discountValue *	Decimal	Decimal(12, 2)	
minAmount *	Decimal	default 0, Decimal(12, 2)	
maxUses	Int?		
usedCount *	Int	default 0	
startsAt	DateTime?		
expiresAt	DateTime?		
isActive *	Boolean	default true	
archivedAt	DateTime?		
description	String?		
perCustomerLimit	Int?		Owner-console targeting & controls
firstBookingOnly *	Boolean	default false	
maxDiscountAmount	Decimal?	Decimal(12, 2)	
stackable *	Boolean	default false	
costBearer *	PromoCostBearer	default SHARED	
appliesToRouteCodes *	String[]	default []	
appliesToOperatorIds *	String[]	default []	
budgetCap	Decimal?	Decimal(12, 2)	
budgetSpent *	Decimal	default 0, Decimal(12, 2)	
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
bookings	Booking	has many	inverse side	—
redemptions	PromotionRedemption	has many	inverse side	—

Indexes: @@index([code, isActive])

PromotionRedemption

One row per successful coupon use. Enables per-customer limits and spend analytics that a single usedCount counter cannot express.

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
promotionId *	String		
bookingId *	String	unique	
customerEmail *	String		
amount *	Decimal	Decimal(12, 2)	
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
promotion	Promotion	has one	promotionId	Restrict
booking	Booking	has one	bookingId	Restrict

Indexes: @@index([promotionId]) @@index([customerEmail])

PlatformConfig

Singleton platform-economics config (id is always "default"). Holds the values the owner console tunes: platform-default commission, optional service fee, traveler multipliers, and the default yield-tier template.

Column	Type	Constraints	Notes
id *	String	PK, default "default"	
commissionRate *	Decimal	default 0.08, Decimal(5, 4)	
serviceFeeType	DiscountType?		
serviceFeeValue *	Decimal	default 0, Decimal(12, 2)	
adultMultiplier *	Decimal	default 1, Decimal(5, 4)	
childMultiplier *	Decimal	default 0.5, Decimal(5, 4)	
infantMultiplier *	Decimal	default 0, Decimal(5, 4)	
defaultPricingTiers	Json?		
updatedBy	String?		
updatedAt *	DateTime	default now(), auto-updated	

Reviews

Review

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
customerId *	String		
bookingId *	String	unique	
scheduleId *	String		
rating *	Int		
text	String?		
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
customer	Customer	has one	customerId	Restrict
booking	Booking	has one	bookingId	Restrict
schedule	Schedule	has one	scheduleId	Restrict

Indexes: @@index([scheduleId]) @@index([customerId])

Reschedule Requests

RescheduleRequest

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
bookingId *	String	unique	
originalLegId *	String		
requestedLegId *	String		
status *	RescheduleStatus	default PENDING	
customerNote	String?		
adminNote	String?		
processedBy	String?		
processedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
booking	Booking	has one	bookingId	Restrict
originalLeg	Leg	has one	originalLegId	—
requestedLeg	Leg	has one	requestedLegId	—

Indexes: @@index([status])

Password Reset

PasswordResetToken

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
email *	String		
tokenHash *	String	unique	
expiresAt *	DateTime		
usedAt	DateTime?		
createdAt *	DateTime	default now()	
Indexes: @@index([email])			

Phase 1B — Facilities, Transit

BoatFacilitiesOnBoat

Column	Type	Constraints	Notes
boatId *	String		
facility *	BoatFacility		

Relations

Relation	Target	Cardinality	Foreign key	On delete
boat	Boat	has one	boatId	Restrict

OperatorServicesOnOperator

Column	Type	Constraints	Notes
operatorId *	String		
service *	OperatorService		

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	Restrict

TransitStop

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
scheduleId *	String		
portName *	String		
arrivalTime *	String		"HH:MM"
departureTime *	String		"HH:MM"
orderIndex *	Int		

Relations

Relation	Target	Cardinality	Foreign key	On delete
schedule	Schedule	has one	scheduleId	Restrict

Indexes: @@index([scheduleId, orderIndex])

Phase 2 — Port, Staff, Holds, Notifications, Kyc

Port

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
shortCode *	String	unique, VarChar(3)	
slug *	String	unique	
name *	String		
island	String?		
province	String?		
latitude	Decimal?	Decimal(9, 6)	
longitude	Decimal?	Decimal(9, 6)	
timezone *	String	default "Asia/Makassar"	
isActive *	Boolean	default true	
deletedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
originSchedules	Schedule	has many	inverse side	—
destinationSchedules	Schedule	has many	inverse side	—

Indexes: @@index([shortCode]) @@index([isActive])

OperatorStaff

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
email *	String	unique	
passwordHash *	String		
fullName *	String		
role *	OperatorStaff Role	default GATE	
status *	OperatorStaff Status	default ACTIVE	
lastLoginAt	DateTime?		
deletedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	Restrict
checkIns	Ticket	has many	inverse side	—
salesBookings	Booking	has many	inverse side	—
cashDrawerSessions	CashDrawerSessions	has many	inverse side	—

Indexes: @@index([operatorId, status])

SeatHold

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
legId *	String		
sessionKey *	String		anonymous session id OR customer id
seats *	Int		
status *	SeatHoldStatus	default ACTIVE	
expiresAt *	DateTime		
bookingId	String?	unique	set when promoted to a real booking
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
leg	Leg	has one	legId	Restrict
booking	Booking	has one (optional)	bookingId	SetNull

Indexes: @@index([legId, status, expiresAt]) @@index([sessionKey])

NotificationLog

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
channel *	NotificationChannel		
type *	NotificationType		
recipient *	String		email / phone
referenceType *	String		'booking' 'refund' 'reschedule'
referenceId *	String		
status *	NotificationStatus	default PENDING	
providerRef	String?		
payloadSummary	Json?		do not store full PII; just enough to debug
sentAt	DateTime?		
failedReason	String?		
attempts *	Int	default 0	
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Indexes: @@index([referenceType, referenceId]) @@index([status, createdAt])

OperatorDocument

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
type *	OperatorDocumentType		
fileUrl *	String		
fileName	String?		
status *	OperatorDocumentStatus	default PENDING	
verifiedBy	String?		Admin.id
verifiedAt	DateTime?		
rejectionNote	String?		
expiresAt	DateTime?		for licences with renewal dates
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	Restrict

Indexes: @@unique([operatorId, type]) @@index([status]) @@index([expiresAt])

Phase 3 — Agents, Loyalty, Fleet, Addons, Weather

Agent

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
email *	String	unique	
passwordHash *	String		
companyName *	String		
contactPerson *	String		
phoneNumber *	String		
tier *	AgentTier	default BRONZE	
commissionRate *	Decimal	default 0.05, Decimal(5, 4)	
isActive *	Boolean	default true	
totalBookings *	Int	default 0	
totalRevenue *	Decimal	default 0, Decimal(14, 2)	
apiKey	String?	unique	
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operators	Operator	has many	inverse side	—
bookings	Booking	has many	inverse side	—

Indexes: @@index([email]) @@index([isActive])

LoyaltyAccount

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
customerId *	String	unique	
pointsBalance *	Int	default 0	
lifetimePoints *	Int	default 0	
tier *	AgentTier	default BRONZE	
tierExpiresAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
customer	Customer	has one	customerId	Cascade
transactions	LoyaltyTransaction	has many	inverse side	—

Indexes: @@index([customerId])

LoyaltyTransaction

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
accountId *	String		
bookingId	String?		
type *	LoyaltyTransactionType		
points *	Int		
description	String?		
expiresAt	DateTime?		
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
account	LoyaltyAccount	has one	accountId	Cascade
booking	Booking	has one (optional)	bookingId	SetNull

Indexes: @@index([accountId, createdAt]) @@index([bookingId])

BoatPosition

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
boatId *	String		
latitude *	Decimal	Decimal(9, 6)	
longitude *	Decimal	Decimal(9, 6)	
speedKnots	Decimal?	Decimal(5, 1)	
heading	Int?		
recordedAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
boat	Boat	has one	boatId	Restrict

Indexes: @@index([boatId, recordedAt])

BookingAddon

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
bookingId *	String		
type *	BookingAddonType		
name *	String		
quantity *	Int	default 1	
unitPrice *	Decimal	Decimal(12, 2)	
totalPrice *	Decimal	Decimal(12, 2)	
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
booking	Booking	has one	bookingId	Restrict

Indexes: @@index([bookingId])

WeatherSnapshot

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
legId *	String		
condition *	WeatherCondition		
waveHeight	Decimal?	Decimal(4, 2)	metres
windSpeedKnots	Decimal?	Decimal(5, 1)	
temperature	Decimal?	Decimal(4, 1)	celsius
source *	String	default "BMKG"	
recordedAt *	DateTime	default now()	
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
leg	Leg	has one	legId	Restrict

Indexes: @@index([legId]) @@unique([legId, recordedAt])

Fx Rates

FxRate

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
currency *	String	VarChar(3)	
rate *	Decimal	Decimal(12, 6)	
fetchAt *	DateTime	default now()	
Indexes: @@index([currency, fetchAt])			

Weather Forecast

WeatherForecast

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
portName *	String		
forecastFor *	DateTime		
waveHeightM	Decimal?	Decimal(4, 2)	
windKnots	Decimal?	Decimal(5, 1)	
condition *	WeatherCondition	default CALM	
fetchAt *	DateTime	default now()	
Indexes: @@index([portName, forecastFor])			

Erp — Sales Channels & Agents

TravelAgent

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
name *	String		
contactPerson	String?		
phoneNumber	String?		
email	String?		
npwp	String?		
address	String?		
defaultCommissionPercent *	Decimal	Decimal(5, 4)	
bankName	String?		
bankAccountNumber	String?		
bankAccountName	String?		
isActive *	Boolean	default true	

Column	Type	Constraints	Notes
deletedAt	DateTime?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	—
bookings	Booking	has many	inverse side	—

Indexes: @@index([operatorId, isActive, deletedAt])

Erp — Cash Drawer

CashDrawerSession

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
staffId *	String		
openedAt *	DateTime		
closedAt	DateTime?		
openingBalance *	Decimal	Decimal(14, 2)	
expectedCash *	Decimal	default 0, Decimal(14, 2)	
closingBalanceCounted	Decimal?	Decimal(14, 2)	
variance	Decimal?	Decimal(14, 2)	
notes	String?		
createdAt *	DateTime	default now()	
updatedAt *	DateTime	auto-updated	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	—
staff	OperatorStaff	has one	staffId	—

Indexes: @@index([operatorId, openedAt]) @@index([staffId, closedAt])

Erp — Operator Notifications

OperatorNotification

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
operatorId *	String		
kind *	NotificationKind		
severity *	NotificationSeverity	default INFO	
title *	String		
body *	String		
actionUrl	String?		
data	Json?		
readAt	DateTime?		
createdAt *	DateTime	default now()	

Relations

Relation	Target	Cardinality	Foreign key	On delete
operator	Operator	has one	operatorId	—

Indexes: @@index([operatorId, readAt, createdAt])

Audit Log

AuditLog

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
entityType *	AuditEntityType		
entityId *	String		
action *	String		
userId	String?		
userRole	AuditUserRole?		
previousState	Json?		
newState	Json?		
ipAddress	String?		
userAgent	String?		
createdAt *	DateTime	default now()	

Indexes: @@index([entityType, entityId]) @@index([createdAt])

LoginAttempt

Column	Type	Constraints	Notes
id *	String	PK, default cuid()	
kind *	LoginKind		
emailKey *	String		
createdAt *	DateTime	default now()	

Indexes: @@index([kind, emailKey, createdAt])

Enumerated types

41 enums. Values are stored as Postgres enum types, so adding one is additive and safe, while removing one that existing rows reference will fail the whole schema push.

Type	Values
PaymentMethod	BANK_TRANSFER, VA_BCA, VA_BNI, VA_BRI, VA_MANDIRI, VA_PERMATA, GOPAY, OVO, DANA, SHOPEEPAY, LINKAJA, QRIS, CREDIT_CARD, PAYPAL
PaymentProvider	MIDTRANS, PAYPAL, XENDIT, MAYAR, STRIPE, DOKU XENDIT — Legacy values retained (unused by the app) so `prisma db push` without --accept-data-loss stays additive: dropping enum values that existing rows still reference is destructive and would fail the whole push, leaving the DB without the columns/values it needs.
RefundReason	CUSTOMER_REQUEST, OPERATOR_CANCELLATION, WEATHER, ADMIN_OVERRIDE
AuditEntityType	BOOKING, PAYMENT, REFUND, LEG, SCHEDULE, BOAT, OPERATOR, CUSTOMER, TICKET, PROMOTION, RESCHEDULE_REQUEST
AuditUserRole	CUSTOMER, OPERATOR, ADMIN, SYSTEM
OperatorStatus	PENDING, ACTIVE, SUSPENDED, REJECTED
AdminRole	SUPER_ADMIN, STAFF
BoatStatus	ACTIVE, MAINTENANCE, INACTIVE
ScheduleStatus	ACTIVE, INACTIVE
LegStatus	OPEN, FULL, SAILED, CANCELLED
BookingStatus	PENDING_PAYMENT, AWAITING_CONFIRMATION, CONFIRMED, CANCELLED_BY_CUSTOMER, CANCELLED_BY_OPERATOR, EXPIRED AWAITING_CONFIRMATION — Paid, but the seat is not promised yet. Operators here work by phone rather than through the dashboard, so an admin rings them to check the boat is actually running before any boarding pass goes out. Seats stay held on the Leg for the whole of this window.
TicketStatus	ISSUED, CHECKED_IN, NO_SHOW, REFUNDED
PaymentStatus	PENDING, SUCCESSFUL, FAILED, EXPIRED
RefundStatus	PENDING, APPROVED, REJECTED, PROCESSING, COMPLETED, FAILED
WebhookEventStatus	PENDING, PROCESSING, PROCESSED, FAILED, SKIPPED
PromoCostBearer	PLATFORM, OPERATOR, SHARED
DiscountType	PERCENT, FLAT
RescheduleStatus	PENDING, APPROVED, REJECTED, CANCELLED
BoatFacility	AIR_CONDITIONING, TOILET, WIFI, LIFE_JACKET, LIFE_RAFT, ENTERTAINMENT_SYSTEM, SNACK_SERVICE, DRINK_SERVICE, CHARGING_PORT, BLANKET, FIRST_AID_KIT, BAGGAGE_COMPARTMENT, OUTDOOR_DECK, CABIN_SEATING, SMOKING_AREA, PRAYER_ROOM
OperatorService	BAGGAGE_INSURANCE, TRAVEL_INSURANCE, HOTEL_TRANSFER, MEAL_INCLUDED, PRIORITY_BOARDING, FLEXIBLE_RESCHEDULE, CHILD_DISCOUNT, GROUP_BOOKING, PET_FRIENDLY, WHEELCHAIR_ACCESSIBLE
FareRefundability	REFUNDABLE, NON_REFUNDABLE, PARTIALLY_REFUNDABLE
ScheduleType	DIRECT, TRANSIT
FareType	REFUNDABLE, NON_REFUNDABLE
OperatorStaffRole	MANAGER, GATE, BOOTH_AGENT, ACCOUNTANT, CAPTAIN, CREW, ENGINEER
OperatorStaffStatus	ACTIVE, SUSPENDED
SeatHoldStatus	ACTIVE, EXPIRED, CONVERTED, RELEASED
NotificationChannel	EMAIL, WHATSAPP, SMS

Type	Values
NotificationType	BOOKING_CONFIRMED, BOOKING_REMINDER_24H, BOOKING_REMINDER_2H, REFUND_APPROVED, REFUND_REJECTED, RESCHEDULE_APPROVED, RESCHEDULE_REJECTED, LEG_CANCELLED, PASSWORD_RESET
NotificationStatus	PENDING, SENT, FAILED, SUPPRESSED
OperatorDocumentType	VESSEL_LICENSE, SIUP, NPWP, INSURANCE_CERTIFICATE, CAPTAIN_LICENSE, OTHER
OperatorDocumentStatus	PENDING, APPROVED, REJECTED, EXPIRED
AgentTier	BRONZE, SILVER, GOLD, PLATINUM
LoyaltyTransactionType	EARNED, REDEEMED, EXPIRED, ADJUSTMENT
BookingAddonType	INSURANCE, HOTEL_TRANSFER, MEAL, BAGGAGE_EXTRA, PRIORITY_BOARDING, OTHER
WeatherCondition	CALM, MODERATE, ROUGH, SEVERE, CLOSED
CurrencyCode	IDR, USD, AUD, EUR, SGD
SalesChannel	GILIFAST, GILIJET, WALK_IN, TRAVEL_AGENT, PHONE, EXTERNAL_AGGREGATOR GILIJET — Pre-rename value for the same thing: our own platform. Existing Booking rows still carry it, and dropping an enum value those rows reference is destructive — `prisma db push` would refuse the whole push. Kept for the same reason as the legacy PaymentProvider values above. Anything asking "was this sold on our platform?" must match both; see DIRECT_SALES_CHANNELS in lib/operator-erp-queries.ts.
EFakturStatus	NOT_REQUIRED, DRAFT, SUBMITTED, ACCEPTED, REJECTED
NotificationKind	CERT_EXPIRY, CASH_VARIANCE, SYNC_FAILURE, PAYROLL_DRAFT_READY, SPT_DRAFT_READY, AGENT_PAYOUT_DUE, PERIOD_CLOSE_PROMPT, CORETAX_REJECTION, BPJS_REJECTION
NotificationSeverity	INFO, WARNING, CRITICAL
LoginKind	OPERATOR, ADMIN, CUSTOMER Per-email login-failure tracking. A short window of recent rows is consulted by lib/login-throttle.ts to deny repeated bad-password attempts. Only failed attempts need to be kept; successful logins clear the window via the helper.